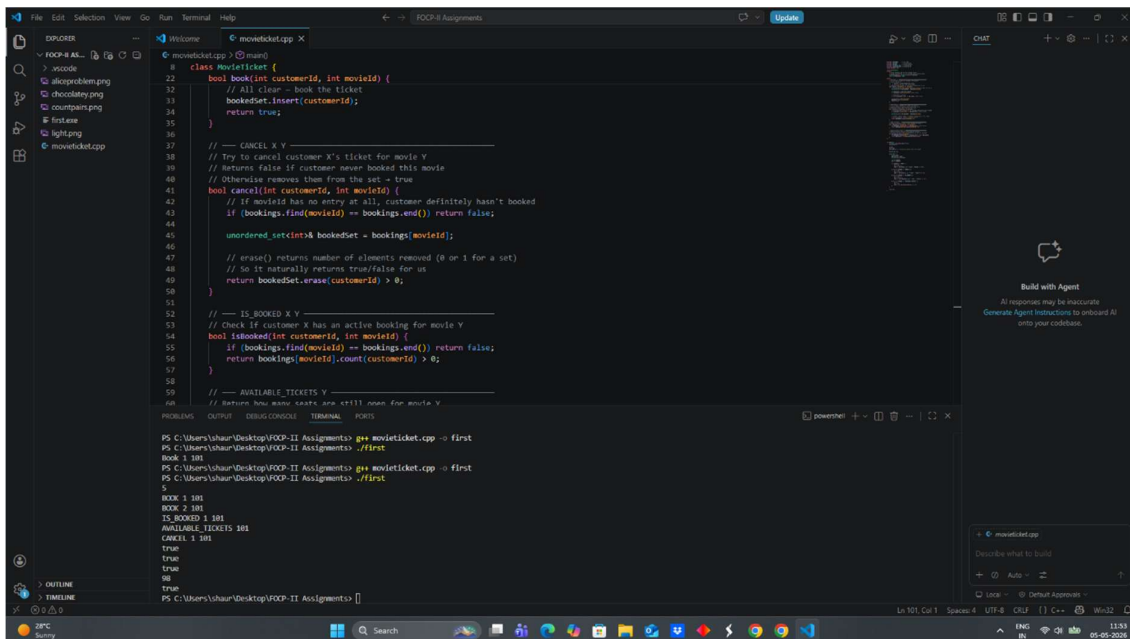
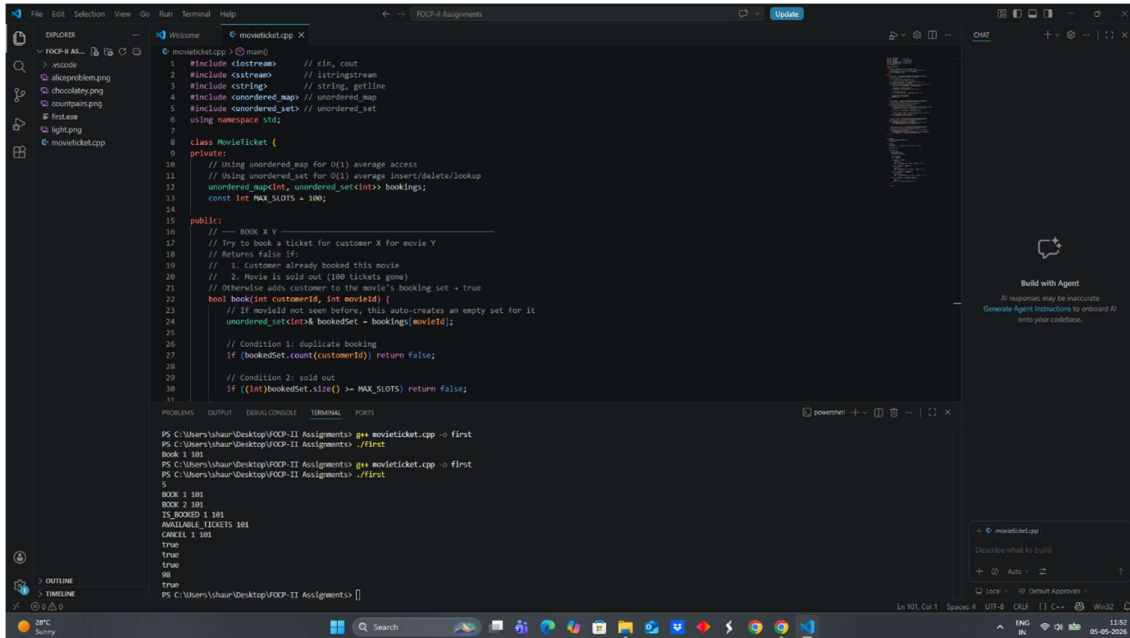




```
1 // movieticketapp.cpp
2 #include <iostream>
3 #include <vector>
4 #include <string>
5 #include <map>
6 #include <unordered_map>
7 #include <algorithm>
8 #include <limits>
9 #include <memory>
10 #include <string_view>
11 #include <chrono>
12 #include <thread>
13 #include <mutex>
14 #include <atomic>
15 #include <future>
16 #include <condition_variable>
17 #include <shared_mutex>
18 #include <weak_ptr>
19 #include <memory_order>
20 #include <atomic_flag>
21 #include <atomic_thread_storage>
22 #include <atomic_int>
23 #include <atomic_intptr>
24 #include <atomic_int64_t>
25 #include <atomic_uint64_t>
26 #include <atomic_int128_t>
27 #include <atomic_uint128_t>
28 #include <atomic_bool>
29 #include <atomic_char>
30 #include <atomic_char16_t>
31 #include <atomic_char32_t>
32 #include <atomic_wchar_t>
33 #include <atomic_int_least<N>>
34 #include <atomic_int_fast<N>>
35 #include <atomic_int_least32_t>
36 #include <atomic_int_fast32_t>
37 #include <atomic_int_least64_t>
38 #include <atomic_int_fast64_t>
39 #include <atomic_uint_least<N>>
40 #include <atomic_uint_fast<N>>
41 #include <atomic_uint_least32_t>
42 #include <atomic_uint_fast32_t>
43 #include <atomic_uint_least64_t>
44 #include <atomic_uint_fast64_t>
45 #include <atomic_uint128_t>
46 #include <atomic_int128_t>
47 #include <atomic_intptr_t>
48 #include <atomic_uintptr_t>
49 #include <atomic_intmax_t>
50 #include <atomic_uintmax_t>
51 #include <atomic_intptr_t>
52 #include <atomic_uintptr_t>
53 #include <atomic_intmax_t>
54 #include <atomic_uintmax_t>
55 #include <atomic_intptr_t>
56 #include <atomic_uintptr_t>
57 #include <atomic_intmax_t>
58 #include <atomic_uintmax_t>
59 // --- AVAILABLE_TICKETS Y
60 // If movie has never been queried/booked, all 100 are available
61 int availableTickets(int movieId) {
62     if (bookings.find(movieId) == bookings.end()) return MAX_SLOTS;
63     return MAX_SLOTS - (int)bookings[movieId].size();
64 }
65
66 };
67
68
69 int main() {
70     ios_base::sync_with_stdio(false);
71     cin.tie(NULL);
72
73     int Q;
74     cin >> Q;
75     cin.ignore(); // consume the newline after the integer
76
77     MovieTicket mt;
78
79     while (Q--) {
80         string line;
81         getline(cin, line);
82         stringstream iss(line);
83
84         string command;
85         iss >> command;
86
87         if (command == "BOOK") {
88             int x, y;
89             iss >> x >> y;
90
91             if (mt.book(x, y) > "true" : "false" << "\n";
92
93         } else if (command == "CANCEL") {
94             int x, y;
95             iss >> x >> y;
96             cout << (mt.cancel(x, y) > "true" : "false" << "\n";
97
98         } else if (command == "IS_BOOKED") {
99             int x, y;
100             iss >> x >> y;
101             cout << (mt.isBooked(x, y) > "true" : "false" << "\n";
102
103         } else if (command == "AVAILABLE_TICKETS") {
104             int y;
105             iss >> y;
106             cout << mt.availableTickets(y) << "\n";
107         }
108     }
109     return 0;
110 }
```

PS C:\Users\shaur\Desktop\FOOP-II Assignments> g++ movieticketapp.cpp -o first
PS C:\Users\shaur\Desktop\FOOP-II Assignments> ./first
BOOK 1 101
PS C:\Users\shaur\Desktop\FOOP-II Assignments> g++ movieticketapp.cpp -o first
PS C:\Users\shaur\Desktop\FOOP-II Assignments> ./first
5
BOOK 1 101
BOOK 2 101
IS_BOOKED 1 101
AVAILABLE_TICKETS 101
CANCEL 1 101
true
true
true
88
true
PS C:\Users\shaur\Desktop\FOOP-II Assignments> |

```
88     int x, y;
89     iss >> x >> y;
90     cout << (mt.book(x, y) > "true" : "false" << "\n";
91
92     } else if (command == "CANCEL") {
93         int x, y;
94         iss >> x >> y;
95         cout << (mt.cancel(x, y) > "true" : "false" << "\n";
96
97     } else if (command == "IS_BOOKED") {
98         int x, y;
99         iss >> x >> y;
100         cout << (mt.isBooked(x, y) > "true" : "false" << "\n";
101
102     } else if (command == "AVAILABLE_TICKETS") {
103         int y;
104         iss >> y;
105         cout << mt.availableTickets(y) << "\n";
106     }
107 }
108
109 return 0;
110 }
```

PS C:\Users\shaur\Desktop\FOOP-II Assignments> g++ movieticketapp.cpp -o first
PS C:\Users\shaur\Desktop\FOOP-II Assignments> ./first
BOOK 1 101
PS C:\Users\shaur\Desktop\FOOP-II Assignments> g++ movieticketapp.cpp -o first
PS C:\Users\shaur\Desktop\FOOP-II Assignments> ./first
5
BOOK 1 101
BOOK 2 101
IS_BOOKED 1 101
AVAILABLE_TICKETS 101
CANCEL 1 101
true
true
true
88
true
PS C:\Users\shaur\Desktop\FOOP-II Assignments> |

Write a C++ program to simulate a movie ticket booking system.

Requirements:

- Use efficient data structures like `unordered_map` and `unordered_set`.
- Each movie can have a maximum of 100 tickets.
- Each customer can book at most one ticket per movie.

Support the following operations:

1. `BOOK XY` → Book a ticket for customer X for movie Y.
 - Return "true" if booking is successful.
 - Return "false" if the customer already booked or movie is full.
2. `CANCEL XY` → Cancel booking of customer X for movie Y.
 - Return "true" if cancellation successful.
 - Return "false" if no booking exists.
3. `IS_BOOKED XY` → Check if customer X has booked movie Y.
 - Return "true" or "false".
4. `AVAILABLE_TICKETS Y` → Return number of available tickets for movie Y.

Input format:

- First line contains integer Q (number of queries).
- Next Q lines contain commands in string format.

Output:

- Print result of each query on a new line.

Additional requirements:

- Use a class-based design (MovieTicket class).
- Use fast I/O (`ios::sync_with_stdio(false)`, `cin.tie(NULL)`).
- Parse input lines using `getline` and `stringstream`.
- Keep the code clean and well-commented.

Here's a clean and efficient C++ implementation that satisfies all your requirements:

```
1 1 // C++
2 2 #include <iostream>
3 3 #include <unordered_map>
4 4 #include <unordered_set>
5 5 #include <string>
6 6 #include <sstream>
```

+ Ask anything

ChatGPT can make mistakes. Check important info. See [Credibility](#) for more.